Sistemas Operacionais

Giovani Delgado Torres

Bruno Bertolini Maluf

Guilherme Ferrari

Guilherme Serafim

Matheus Belloto

Permissões, Usuários e Grupos no Linux

Sorocaba/SP
2026

SUMÁRIO

1 INTRODUCAO 3

2 OBJETIVO DO EXPERIMENTO 3

3 MATERIAIS UTILIZADOS 3

4 PROCEDIMENTO EXPERIMENTAL 3

5 RESULTADOS 3

6 CONCLUSAO 9

REFERÊNCIAS 10

1 INTRODUCAO

Esta lista de exercícios foi proposta com o objetivo de entender melhor como funcionam as permissões de arquivos e diretórios no sistema operacional Linux, além da relação entre usuários e grupos.

Durante a atividade, trabalhamos diretamente no terminal, testando comandos e observando na prática como o sistema reage a diferentes tipos de permissões. Isso ajudou bastante a visualizar como o Linux garante a segurança dos arquivos e evita acessos indevidos.

2 OBJETIVO DO EXPERIMENTO

O principal objetivo foi aprender a controlar permissões de arquivos e diretórios utilizando comandos como chmod, chown, chgrp e sudo.

Além disso, a atividade também buscou mostrar como o sistema diferencia usuários comuns do superusuário (root), e como isso impacta no acesso aos recursos do sistema.

3 MATERIAIS UTILIZADOS

Foi utilizado um computador com Windows, rodando o Ubuntu através do WSL.

Os comandos foram executados diretamente no terminal Linux (bash), utilizando uma conta de usuário comum e, quando necessário, o uso do sudo para acessar funções administrativas.

4 PROCEDIMENTO EXPERIMENTAL

Os exercícios foram feitos no terminal, seguindo a ordem proposta (nível 1 ao nível 5). Para cada atividade, executamos os comandos pedidos e analisamos o que acontecia, principalmente quando ocorria algum erro de permissão.

- Nível 1: Permissões Básicas

1.

```
$ touch relatorio.txt
$ ls -l
```

Ao executar o comando, foi possível observar algo como -rw-r--r--.
Esse conjunto de letras mostra quem pode ler, escrever ou executar o arquivo. No caso, o dono pode ler e escrever, enquanto os outros só podem ler.

2.

```
$ mkdir Lab_Engenharia
$ ls -l
```

O diretório aparece com um d no início (drwxr-xr-x).
Diferente dos arquivos, diretórios precisam da permissão de execução (x) para permitir acesso ao conteúdo.

3.

```
$ ls -l
```

Nesse comando também conseguimos ver quem é o dono do arquivo e a qual grupo ele pertence.

4.

```
$ chmod g+w relatorio.txt
```

Aqui foi adicionada permissão de escrita para o grupo.

5.

```
$ chmod o-r Lab_Engenharia
```

Removemos a permissão de leitura para outros usuários.

6.

Ao tentar criar um arquivo em um local sem permissão:

```
$ touch teste.txt
```

O sistema retorna: **Permission denied**, indicando que não temos autorização.

Nível 2: Modo Octal

7.

```
$ chmod 755 setup.sh
```

O dono fica com permissão total (rwx), e grupo/outros apenas leitura e execução.

8.

```
$ chmod 644 arquivo.txt
```

O arquivo não pode ser executado, apenas lido e editado pelo dono.

9.

```
$ chmod 600 arquivo.txt
```

Arquivo totalmente privado, só o dono acessa.

10.

```
$ chmod 000 arquivo.txt
```

Ninguém tem acesso. Nem o dono consegue abrir normalmente.

11.

```
$ chmod 770 pasta
```

Somente dono e grupo conseguem acessar. Outros ficam bloqueados.

Nível 3: Usuários e sudo

12.

```
$ whoami
$ sudo whoami
```

O primeiro mostra o usuário atual, o segundo mostra **root**.

13.

```
$ su
$ su -
```

O su - carrega o ambiente completo do usuário.

14.

```
$ cat /etc/shadow
$ sudo cat /etc/shadow
```

Sem sudo não funciona, pois é um arquivo extremamente sensível.

15.

```
$ id
```

Mostra os grupos do usuário.

16.

```
$ chgrp docentes arquivo
```

Altera o grupo do arquivo.

17.

```
$ sudo chown root relatorio.txt
```

Depois disso, não conseguimos mais editar o arquivo normalmente.

Nível 4: Prática

18.

```
$ chmod +x sensor.sh
$ ./sensor.sh
```

Permite executar o script.

19.

```
$ chmod -x pasta
$ ls pasta
```

Sem permissão de execução, não conseguimos acessar o conteúdo da pasta

20. Sticky Bit

O diretório /tmp usa essa configuração para impedir que um usuário apague arquivos de outro.

21.

```
$ umask
```

Define as permissões padrão de novos arquivos.

22.

```
$ chmod -R 755 pasta
```

Aplica permissões em tudo dentro da pasta.

23.

```
$ which chmod
$ ls -l /bin/chmod
```

Mostra onde está o comando e suas permissões.

Nível 5: Casos Reais

24.

Se um processo tenta escrever sem permissão, o sistema simplesmente bloqueia a ação.

25.

```
$ chmod 700 backup
```

Apenas o dono consegue acessar o backup.

26.

Depois de transferir para root, não é possível recuperar sem sudo.

27.

```
$ find /home -perm 777
```

Busca arquivos com permissão total (insegura).

28.

Os arquivos de /var/log são protegidos porque contêm informações importantes do sistema.

29.

```
$ chown outro arquivo
```

Erro: Operation not permitted

30. Conclusão técnica

O comando sudo permite executar ações como administrador, pois utiliza o modo supervisor do sistema operacional, que tem controle total e não é limitado pelas permissões normais.

5 RESULTADOS

Com a realização dos exercícios, ficou bem claro como o Linux controla o acesso aos arquivos. Pequenas mudanças nas permissões já fazem bastante diferença no que pode ou não ser feito.

Também foi possível perceber que o usuário root tem total controle do sistema, enquanto usuários comuns possuem várias restrições, o que é essencial para manter a segurança.

6 CONCLUSAO

A atividade ajudou muito a entender como funcionam as permissões no Linux na prática, não só na teoria. Testar os comandos e ver os erros acontecendo foi importante para fixar o conteúdo.

Além disso, ficou evidente como o sistema operacional é organizado em relação à segurança, principalmente com a separação entre usuário comum e root.

Esse tipo de conhecimento é muito importante para quem pretende trabalhar com tecnologia, principalmente na área de sistemas e infraestrutura.

REFERÊNCIAS

GNU PROJECT. Documentação oficial do Linux.
TANENBAUM, Andrew S. Sistemas Operacionais Modernos.